

AGEX Developer Guide

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX를 기반으로 Agent, Workflow, Plugin, Knowledge Integration, Memory Integration 및 Application을 실제로 개발하는 Software Developer, AI Engineer, Platform Engineer, Integration Engineer를 위한 공식 개발 가이드이다.

본 문서는 AGEX Product Documentation과 API Guide를 전제로 하며, 개발자가 다음 작업을 수행하는 방법을 설명한다.

- 개발 환경을 구성한다.
- AGEX SDK 또는 API Client를 초기화한다.
- Agent를 정의하고 실행한다.
- Workflow를 작성하고 검증한다.
- Knowledge를 연결한다.
- Memory Policy를 설정한다.
- Plugin을 개발한다.
- Plugin Action을 Agent Tool로 노출한다.
- Model Provider Adapter를 개발한다.
- Event/Webhook Integration을 구현한다.
- Local Runtime에서 테스트한다.
- Contract Test와 AI Evaluation을 작성한다.
- Production Publish 전 검증한다.
- Application Layer에서 AGEX를 안전하게 호출한다.

본 문서는 특정 Application이나 산업을 전제로 하지 않는다.

2. Developer Guide의 공식 정의

AGEX Developer Guide는 개발자가 AGEX의 공식 Resource Contract와 Runtime

Boundary를 준수하면서 Agent·Workflow·Knowledge·Memory·Plugin·Model 및 Application Integration을 구현하기 위한 공식 개발 기준이다.

3. 개발자가 먼저 이해해야 할 핵심 개념

AGEX 개발자는 최소 다음 개념을 이해해야 한다.

1. Tenant
2. Resource
3. Version / Revision
4. Execution
5. Runtime
6. Agent
7. Workflow
8. Knowledge
9. Memory
10. Plugin
11. Model Gateway
12. Permission / Policy

이 개념을 우회하여 직접 구현하는 방식은 AGEX 개발 방식이 아니다.

4. 개발 기본 원칙

AGEX 개발은 다음 원칙을 따른다.

- Contract First
- Schema First
- API First
- Runtime First

- Tenant Context Everywhere
- Security by Default
- Least Privilege
- Version-Pinned Execution
- Provider Abstraction
- Explicit Side Effects
- Structured Output
- Bounded Autonomy
- Idempotent Side Effects
- Observable by Default
- Test Before Publish
- No Direct Core Database Access
- No Provider SDK Inside Agent Logic
- No Secret in Prompt
- No Raw Chain-of-Thought Dependency
- No Hidden Cross-Tenant State

5. 개발 환경 구성

개발자는 가능한 한 Production과 동일한 Contract를 사용하는 Local Environment를 사용한다.

권장 구성:

Developer Machine

└─ AGEX API

└─ Local Runtime

└─ Database

- └─ Queue
- └─ Cache
- └─ Object Storage
- └─ Mock Model Provider
- └─ Mock Plugin Runtime

Local Environment는 Production 규모를 복제하는 것이 아니라 동일한 동작 의미를 제공하는 것이 목적이다.

6. Repository 기본 구조

예시 논리 구조:

agex/

- └─ apps/
 - | └─ api/
 - | └─ console/
 - | └─ worker/
 - | └─ cli/
- └─ packages/
 - | └─ core/
 - | └─ runtime/
 - | └─ agent/
 - | └─ workflow/
 - | └─ knowledge/
 - | └─ memory/
 - | └─ plugin/

```
|   |— model/
|   |— security/
|   └─ sdk/
|
|— schemas/
|— tests/
|— infrastructure/
└─ docs/
```

실제 Repository 구조가 달라도 Domain Boundary는 유지한다.

7. 개발 전 체크

새 Feature 개발 전 최소 다음을 확인한다.

- 어느 Domain에 속하는가.
 - Resource인가.
 - API가 필요한가.
 - Event가 필요한가.
 - Tenant-owned인가.
 - 어떤 Permission이 필요한가.
 - Side Effect가 있는가.
 - 어떤 Versioning이 필요한가.
 - 어떤 Audit가 필요한가.
 - 어떤 Test가 필요한가.
-

8. Domain Boundary

한 Domain이 다른 Domain의 Database Table을 직접 읽는 방식은 사용하지 않는다.

예:

Agent Module

→ Memory Repository SQL 직접 조회

대신:

Agent Module

→ Memory Service Contract

를 사용한다.

9. Canonical Schema

AGEX의 Public Contract는 가능한 한 Canonical Schema에서 시작한다.

Schema는 다음에 재사용될 수 있다.

- API
- SDK
- Event
- Resource Validation
- Plugin Manifest

SDK Class를 Canonical Source of Truth로 사용하지 않는다.

10. Resource 개발 순서

새 Resource Type 개발 시 권장 순서:

Domain Definition

→ Schema

→ Validation

→ Lifecycle

→ Repository

→ Service Contract

→ API

→ Events

→ Audit

→ Tests

→ SDK

→ Documentation

11. Resource 공통 필드

새 Resource는 필요한 경우 다음 표준을 따른다.

id

type

tenant_id

name

version

revision

state

metadata

created_at

updated_at

모든 Resource가 무조건 동일 필드를 가져야 하는 것은 아니지만 기존 공통 Contract와 일관성을 유지한다.

12. Specification과 Status

Resource가 선언적 정의와 운영 상태를 모두 가질 경우 분리한다.

예:

```
{  
  "specification": {  
    "goal": "...",  
    "model_profile_id": "mdl_1"  
  },  
  "status": {  
    "health": "healthy"  
  }  
}
```

사용자가 변경하는 Desired Configuration과 Runtime이 변경하는 Observed State를 혼합하지 않는다.

13. Revision

Draft 수정 시 Revision을 증가시킨다.

Concurrent Update를 방지하기 위해 ETag 또는 Revision Check를 사용한다.

14. Published Version

Publish된 Resource Version은 Immutable하게 취급한다.

다음 방식은 허용하지 않는다.

Agent v3

→ production hotfix by direct update

수정이 필요하면 v4를 생성한다.

15. Dependency Pinning

Published Resource는 가능한 한 Dependency Version을 명시적으로 고정한다.

예:

Workflow v4

→ Agent v7

→ Plugin v2

암묵적인 latest Dependency 사용을 최소화한다.

16. SDK Client 초기화

개념 예:

```
const agex = new AGEXClient({  
  endpoint: process.env.AGEX_ENDPOINT,  
  credentialProvider,  
  tenantId: "ten_123"  
});
```

또는 Python:

```
agex = AGEXClient(  
    endpoint=endpoint,  
    credential_provider=credential_provider,  
    tenant_id="ten_123",  
)
```

실제 SDK Signature는 공식 SDK Reference를 따른다.

17. Credential 관리

개발자는 Source Code에 다음을 하드코딩하지 않는다.

```
API_KEY = "..."
```

대신:

- Environment Secret
- Workload Identity
- Secret Manager

를 사용한다.

18. Tenant Context

모든 Tenant Resource 호출에는 Tenant Context를 명확히 유지한다.

Application에서 User Input으로 Tenant ID를 그대로 신뢰하지 않는다.

현재 인증된 Identity가 해당 Tenant에 속하는지 서버가 확인해야 한다.

19. Agent 개발 개요

Agent 개발 순서:

Goal

→ Instruction

→ Capability

→ Model Policy

→ Knowledge Binding

→ Memory Policy

→ Tool Binding

→ Permission

→ Autonomy

→ Evaluation

→ Publish

Agent Prompt부터 먼저 작성하는 방식보다 전체 실행 Contract를 먼저 정의한다.

20. Agent Definition 예

개념 예:

name: example-agent

goal:

사용자의 요청을 분석하고 허용된 도구를 사용해 작업을 수행한다.

autonomy_level: L1

model_policy:

profile_id: model-profile-default

knowledge:

collections:

- collection-main

memory:

read:

- session

write:

- proposal

tools:

- item.lookup

이는 개념 예이며 실제 Schema는 공식 Contract를 따른다.

21. Agent Goal 작성

좋은 Goal:

- 결과 중심
- 역할 명확
- 검증 가능
- 구현 세부와 분리

나쁜 예:

Model A를 3번 호출하고 Plugin B의 /v2 API를 실행하라.

이런 내용은 Agent Goal보다 Workflow 또는 Tool Contract에 적합하다.

22. Instruction Hierarchy

Agent 개발자는 Instruction 우선순위를 이해해야 한다.

기본 우선순위:

Platform Security Policy

→ Platform Execution Policy

→ Tenant Policy

→ Agent Permission

→ Agent Core Instruction

→ Workflow Instruction

→ User Task Instruction

→ Runtime Context

User Prompt가 상위 Policy를 Override할 수 없도록 한다.

23. Agent Capability

Capability Manifest에는 Agent가 수행할 수 있는 기능 범위를 선언한다.

예:

- knowledge.retrieve
- memory.recall
- plugin.call
- workflow.start
- agent.delegate

선언하지 않은 Capability를 Runtime에서 임의로 사용하지 않는다.

24. Agent Permission

Capability와 Permission은 다르다.

예:

Agent가 plugin.call Capability를 지원하더라도 특정 Plugin Action에 대한 Permission이 없으면 실행할 수 없다.

Capability

≠

Authorization

25. Model Policy

Agent는 특정 Provider SDK를 직접 사용하지 않는다.

대신 Model Policy를 설정한다.

예:

model_policy:

 required_capabilities:

- structured_output
- tool_calling

allowed_trust:

- Private
 - Direct Approved
-

26. Context Builder

Agent 개발자는 Prompt 문자열을 직접 무제한 누적하기보다 Context Builder를 사용한다.

Context Candidate:

- System Context
 - Agent Instruction
 - User Input
 - Knowledge
 - Memory
 - Tool Results
 - Workflow Context
-

27. Context Budget

Context Window에는 제한이 있으므로 우선순위를 적용한다.

예:

Security/System

>

Current Task

>

Required Knowledge

>

Relevant Memory

>

Historical Context

저우선순위 정보를 무제한 추가하지 않는다.

28. Knowledge Binding

Agent는 Knowledge Collection ID 또는 Knowledge Policy를 통해 검색 범위를 정의한다.
전체 Knowledge를 Prompt에 미리 주입하지 않는다.

29. Knowledge Retrieval 호출

개념:

```
const results = await agex.knowledge.retrieve({  
  collectionIds: ["knc_1"],  
  query: "runtime recovery",  
  limit: 5  
});
```

Agent Runtime 내부에서는 Tenant/Permission Filter가 자동 적용되어야 한다.

30. Citation 처리

Knowledge 기반 결과를 최종 응답에 사용해야 하는 Agent는 Retrieval Result의 Citation 정보를 유지하도록 설계한다.

중간 단계에서 Source Reference를 버리지 않는다.

31. Memory Read Policy

Agent가 사용할 수 있는 Memory Scope를 제한한다.

예:

memory:

read:

- session
- agent

User Memory가 필요하지 않은 Agent에게 User Memory 전체 접근을 부여하지 않는다.

32. Memory Write

Agent가 생성한 정보를 바로 Long-Term Memory에 저장하지 않는다.

권장:

Agent

→ Memory Proposal

→ Policy

→ Store

33. Memory Candidate 예

구조화된 형태를 사용한다.

{

"type": "semantic",

"scope": "user",

"content": "...",

"confidence": 0.91,


```
"reason": "..."  
}
```

실제 저장은 Memory Engine이 결정한다.

34. Tool Binding

Agent Tool은 Native Tool 또는 Plugin Action Projection일 수 있다.

개념 구조:

Plugin Package

→ Capability

→ Action

→ Agent Tool

Agent 코드가 Plugin의 내부 구현을 알아서는 안 된다.

35. Tool Schema

Tool에는 명확한 Input/Output Schema가 필요하다.

예:

```
{  
  "name": "item.lookup",  
  "input_schema": {  
    "type": "object",  
    "properties": {  
      "item_id": {  
        "type": "string"  
      }  
    }  
  },  
}
```

```
"required": ["item_id"]  
}  
}
```

36. Tool Result

Tool Result도 Untrusted External Data일 수 있다.

Tool 결과 문자열을 새로운 System Instruction처럼 처리하지 않는다.

37. Tool Side Effect

Agent Tool 개발 시 반드시 Side Effect Classification을 선언한다.

- Read Only
 - Reversible Write
 - Irreversible Write
 - Privileged Action
-

38. High-Risk Tool

Irreversible 또는 Privileged Action은 추가적으로 다음을 고려한다.

- Approval
 - Stronger Permission
 - Audit
 - Idempotency
 - Compensation
-

39. Action Validation

표준 흐름:

Model proposes Tool Call

- Schema Validation
- Capability
- Permission
- Policy
- Side Effect
- Approval
- Runtime Execute

Model Output을 직접 External API에 연결하지 않는다.

40. Structured Output

Agent가 후속 시스템 처리에 사용할 값을 생성하는 경우 가능한 한 Structured Output을 사용한다.

예:

```
{  
  "decision": "approve",  
  "confidence": 0.88  
}
```

문자열 Parsing에 의존하지 않는다.

41. Agent Loop

기본:

Observe

- Build Context
- Evaluate

→ Plan

→ Select Action

→ Validate

→ Execute

→ Evaluate Result

→ Complete / Continue

Loop에는 반드시 제한이 있어야 한다.

42. Agent Loop Limit

최소 다음 중 일부를 사용한다.

- Max Iterations
- Max Duration
- Max Cost
- Max Tool Calls
- Max Child Executions

무제한 while true Agent Loop를 만들지 않는다.

43. Agent Completion

Agent는 명확한 종료 상태를 반환해야 한다.

예:

- completed
- needs_input
- approval_required
- failed

단순 Model 응답이 비어 있지 않다는 이유로 Completed로 간주하지 않는다.

44. Agent Failure

Failure는 구조화한다.

예:

```
{  
  "code": "TOOL_EXECUTION_FAILED",  
  "retryable": true  
}
```

모든 실패를 Model에게 재질문하여 해결하려 하지 않는다.

45. Agent Test

최소 테스트:

- Basic Success
 - Invalid Input
 - Model Failure
 - Tool Permission Denied
 - Tool Failure
 - Knowledge Empty
 - Memory Unavailable
 - Cost/Iteration Limit
-

46. Agent Evaluation

Software Test와 별개로 평가한다.

평가 후보:

- Task Success

- Tool Correctness
 - Groundedness
 - Policy Compliance
 - Cost
 - Latency
-

47. Agent Regression

Prompt, Model Profile, Tool Set, Knowledge 변경 시 Baseline과 Candidate를 비교한다.
평가 실패 시 자동 Production Promote하지 않는다.

48. Workflow 개발 개요

Workflow 개발 순서:

Input Schema

→ Steps

→ Transitions

→ Variable Scope

→ Error Policy

→ Retry/Timeout

→ Approval

→ Output Schema

→ Tests

→ Publish

49. Workflow Definition 예

개념:

name: example-workflow

input_schema:

type: object

steps:

- id: analyze

type: agent

- id: decide

type: condition

- id: execute

type: plugin

- id: end

type: end

50. Workflow Graph Validation

Publish 전 최소 검증:

- Start 존재
- Reachable End 존재
- 잘못된 Edge 없음
- Cycle 규칙 준수
- Missing Dependency 없음

- Variable Mapping 유효
 - Loop Limit 존재
-

51. Step Input Mapping

Step 입력은 명시적으로 Mapping한다.

예:

input:

item_id: \${workflow.input.item_id}

이전 Step의 모든 Output을 자동으로 다음 Step에 노출하지 않는다.

52. Variable Scope

구분:

- Workflow
- Branch
- Loop
- Step
- Secret

Secret 값을 Workflow 일반 변수로 복사하지 않는다.

53. Agent Step

설정:

type: agent

agent_id: agt_123

version: 4

Workflow의 Permission이 Agent에게 전체 상속되지 않는다.

54. Plugin Step

Plugin Action을 명시한다.

type: plugin

action: item.update

실행 시 설치 상태와 Permission이 다시 검증된다.

55. Model Step

간단한 AI Task에는 Agent 없이 Model Step을 사용할 수 있다.

적합:

- 분류
- 구조화 추출
- 짧은 변환

복잡한 계획/도구 사용에는 Agent가 적합할 수 있다.

56. Condition Step

Condition은 가능한 한 deterministic하게 작성한다.

예:

```
result.status == "approved"
```

자연어 판단이 필요한 경우 이전 Model/Agent Step에서 구조화 결과를 생성한다.

57. Parallel

Parallel Branch를 사용할 때 Merge Rule을 명시한다.

지원 개념:

- All

- Any
 - Quorum
 - First Successful
-

58. Join

다수 Branch의 동일 Key가 충돌할 경우 명시적 Merge Strategy를 사용한다.

last write wins를 암묵적으로 사용하지 않는다.

59. Loop

반드시 종료 조건 또는 Max Iteration을 설정한다.

예:

```
max_iterations: 10
```

60. Wait

장기간 Wait는 Worker Sleep으로 구현하지 않는다.

Runtime Durable Wait를 사용한다.

61. Event Wait

Correlation Key를 명확히 정의한다.

예:

```
event_type = external.completed
```

```
correlation_id = workflow.instance_id
```

62. Approval Step

Approval Payload를 구체적으로 구성한다.

승인 대상이 불명확한 “계속 진행할까요?” 형태보다 실제 Action과 Target을 포함한다.

63. Sub-Workflow

다른 Workflow를 호출할 때 Version을 Pin한다.

Parent Workflow Permission이 Child Workflow에 자동 전파되지 않는다.

64. Compensation

Side Effect가 있는 Workflow에서는 보상 작업을 설계할 수 있다.

예:

create reservation

→ later failure

→ cancel reservation

65. Workflow Retry

Retry는 Step Type별로 다르게 설정한다.

예:

- Deterministic Function: 제한적 Retry
 - External API: Backoff Retry
 - Permission Denied: No Retry
-

66. Workflow Checkpoint

장기 Workflow는 중요한 경계에서 Checkpoint를 생성할 수 있다.

Recovery 시 모든 Step을 처음부터 재실행하지 않도록 한다.

67. Workflow Test

최소:

- Happy Path
- Branch A/B
- Approval
- Retry
- Timeout
- Cancellation
- Recovery
- Compensation

68. Dynamic Workflow

AI가 Workflow Candidate를 생성하는 기능은 Advanced Capability다.

생성 가능 범위:

- 등록된 Step Type
- 허용된 Agent
- 허용된 Plugin
- Budget
- Depth
- Loop Limit

Arbitrary Code Generation/Execution과 동일하게 취급하지 않는다.

69. Plugin 개발 개요

Plugin 개발 절차:

Define Package

→ Manifest

- Capabilities
 - Actions
 - Schemas
 - Permissions
 - Network
 - Runtime
 - Tests
 - Sign
 - Publish
-

70. Plugin Manifest 개념 예

plugin:

id: example.plugin

version: 1.0.0

capabilities:

- items

actions:

- name: items.get

side_effect: read_only

71. Plugin Action 설계

좋은 Action은 단일 목적을 가진다.

좋은 예:

items.get

items.create

items.update

나쁜 예:

execute_arbitrary_request

72. Plugin Input Validation

Plugin Runtime 진입 전 Input Schema Validation을 수행한다.

외부 시스템이 다시 Validation하더라도 AGEX Validation을 생략하지 않는다.

73. Plugin Output Validation

외부 API Response를 그대로 Agent에게 전달하기보다 Output Schema를 검증한다.

잘못된 Output은 Plugin Error로 처리한다.

74. Credential Broker

Plugin은 Credential Reference를 받아 Runtime에서 Resolve한다.

개념:

Plugin Action

→ Credential Broker

→ External Request

Agent/Workflow는 실제 Secret을 볼 필요가 없다.

75. Network Manifest

Plugin이 필요한 Network를 선언한다.

예:

network:

allow:

- api.example.com:443

실행 시 실제 Egress Policy와 교차 검증한다.

76. Plugin Idempotency

Write Action은 가능한 경우 Idempotency Key를 External API에 전달하거나 자체 Side-Effect Ledger를 사용한다.

Queue Redelivery에서 중복 쓰기를 방지해야 한다.

77. Plugin Timeout

모든 외부 호출에 명시적 Timeout을 설정한다.

기본 HTTP Library Timeout 없음 상태를 사용하지 않는다.

78. Plugin Retry

HTTP 400/403과 Network Timeout을 동일하게 Retry하지 않는다.

Error Mapping을 정의한다.

79. Plugin Async Action

외부 작업이 오래 걸리는 경우:

start external operation

→ external operation id

→ wait/callback/poll

방식을 사용한다.

Plugin Worker가 수십 분 Blocking되지 않게 한다.

80. Plugin Webhook

외부 Callback을 받을 경우 다음을 확인한다.

- Signature
- Correlation
- Replay
- Tenant
- Action Instance

81. Plugin Sandbox

Untrusted Plugin은 Sandbox Contract를 따라야 한다.

제한:

- CPU
- Memory
- Filesystem
- Network
- Time

82. Plugin SDK

Plugin SDK는 다음 기능을 제공할 수 있다.

- Manifest Types
- Action Handler
- Context
- Secret Reference
- Logging

- Error Mapping
- Test Harness

SDK를 사용해도 서버 Runtime Security가 최종 Enforcement를 담당한다.

83. Plugin Test

최소:

- Manifest Validation
 - Input Validation
 - Output Validation
 - Credential Missing
 - Network Denied
 - Timeout
 - Retry
 - Idempotency
 - Tenant Isolation
-

84. Plugin Packaging

Package에는 다음이 포함될 수 있다.

- Manifest
- Code Artifact
- Schemas
- Metadata
- Signature

Published Package Version은 변경하지 않는다.

85. Model Provider Adapter 개발

새 Provider를 추가할 때 Agent 코드를 수정하지 않는다.

Provider Adapter Interface를 구현한다.

86. Model Provider Interface

개념:

listModels()

capabilities()

generate()

stream()

embed()

rerank()

health()

getUsage()

Provider가 지원하지 않는 Capability는 명확히 unsupported로 반환한다.

87. Generate Contract

공통 Input:

- Messages / Context
- Output Schema
- Tool Definitions
- Temperature or policy parameters
- Limits

공통 Output:

- Content

- Structured Data
 - Tool Calls
 - Usage
 - Finish Reason
 - Provider Metadata
-

88. Streaming Adapter

Provider별 Stream Event를 AGEX Common Stream Event로 Normalize한다.

Client가 Provider별 Event Format을 알 필요가 없어야 한다.

89. Structured Output Adapter

Provider의 Native Schema 기능이 없을 경우 Emulation이 필요할 수 있다.

단, "JSON처럼 보이는 텍스트"를 Valid Structured Output으로 간주하지 않는다.

Schema Validation을 수행한다.

90. Tool Calling Adapter

Provider별 Tool Calling 표현을 공통 Action Candidate 형식으로 변환한다.

Tool 실행 자체는 Adapter가 하지 않는다.

91. Usage Normalization

Provider별 Usage Field를 공통 Meter로 변환한다.

예:

- input tokens
- output tokens
- cache tokens

- requests

92. Provider Error Mapping

Provider Error를 공통 Error Category로 변환한다.

예:

- RATE_LIMITED
- AUTH_FAILED
- MODEL_UNAVAILABLE
- CONTENT_REJECTED
- TIMEOUT

93. Provider Health

Health Check는 실제 Production Request와 동일 경로를 과도하게 소비하지 않도록 설계한다.

Provider 상태는 Router에 전달한다.

94. Private Model Adapter

Private Model Runtime도 동일 Provider Interface를 구현한다.

Agent 입장에서는 Direct Provider와 사용 방식이 동일해야 한다.

95. Knowledge Connector 개발

Connector Interface 예:

connect()

healthCheck()

list()

fetch()

getMetadata()

getChanges()

96. Connector 원칙

Connector는 Source에서 원본을 가져오는 책임을 가진다.

Chunking, Embedding, Ranking을 Connector 안에 넣지 않는다.

이들은 Knowledge Engine 책임이다.

97. Incremental Sync

가능하면 Source의 변경사항만 처리한다.

필요 정보:

- source item id
 - version/etag
 - updated timestamp
 - deleted state
-

98. Connector Credential

Plugin과 동일하게 Secret Reference를 사용한다.

Connector Config에 평문 Password를 저장하지 않는다.

99. Parser 개발

Parser는 입력 Format을 Normalized Document Model로 변환한다.

출력:

- Text

- Structure
- Metadata
- Media References

Embedding을 Parser가 직접 수행하지 않는다.

100. Knowledge Metadata

가능하면 다음을 유지한다.

- Source ID
- Document ID
- Source Version
- Title
- Timestamp
- Classification
- ACL

Citation을 위해 Provenance를 잃지 않는다.

101. Retrieval Extension

Custom Reranker나 Retriever를 만들 경우 공통 Retrieval Contract를 구현한다.

Security Filter 이후 Candidate를 처리하도록 한다.

102. Memory Provider 확장

External Memory Store를 사용할 경우 AGEX Memory Contract를 유지한다.

Provider 구현이 Tenant Scope, Delete, Retention을 지원하지 않으면 Production Adapter로 적합하지 않을 수 있다.

103. Memory Deletion Contract

Delete가 요청되면 Provider가 실제 Retrieval에서 제거되었음을 보장해야 한다.

Eventually Consistent한 경우 상태를 추적해야 한다.

104. Event 개발

Domain Event는 과거에 발생한 사실을 표현한다.

좋은 예:

AgentPublished

ExecutionCompleted

PluginInstalled

나쁜 예:

PleaseDoSomething

Command와 Event를 구분한다.

105. Event Schema

Event에는 최소:

- event_id
- event_type
- version
- tenant_id where applicable
- timestamp
- resource reference

를 포함한다.

106. Event Payload

전체 Resource Dump보다 필요한 변경 정보와 Reference를 전달하는 방식을 우선한다.
민감정보가 Event Bus에 과도하게 복제되지 않도록 한다.

107. Event Consumer

Consumer는 다음을 전제로 한다.

- Duplicate Event 가능
- Out-of-order 가능
- Delayed Delivery 가능

Idempotency와 Version Check를 구현한다.

108. Webhook Consumer 개발

Webhook Receiver 기본 구조:

Receive

→ Verify Signature

→ Verify Timestamp

→ Deduplicate Event

→ Enqueue Internal Work

→ Return 2xx

긴 처리를 Webhook HTTP Request 안에서 끝내려 하지 않는다.

109. Application Integration

Application은 AGEX를 다음과 같이 사용한다.

Application UI

→ Application Backend

→ AGEX API

→ Runtime

고권한 AGEX Credential을 Browser에 직접 넣지 않는다.

110. Application Backend

Application Backend는 다음을 담당한다.

- 사용자 요청 수신
- 자신의 Domain Logic
- AGEX API 호출
- Execution ID 저장
- 상태 표시

AGEX Core Database를 직접 Query하지 않는다.

111. Agent 실행 Integration

개념:

```
const execution = await agex.agents.execute(agentId, {  
  version: 4,  
  input: { message: userInput },  
  idempotencyKey  
});
```

이후:

```
const state = await agex.executions.get(execution.id);
```

112. Long-Running UX

Application은 다음 상태를 사용자에게 표현할 수 있어야 한다.

- queued

- running
- waiting
- waiting_for_approval
- completed
- failed

running과 "웹 요청 로딩 중"을 동일하게 처리하지 않는다.

113. Streaming Integration

Streaming을 사용할 경우 다음 두 State를 분리한다.

Stream Connection State

Execution State

Stream이 끊어지면 Execution API로 복구한다.

114. Cancellation UX

사용자가 취소 버튼을 눌렀을 때 HTTP 연결만 끊지 않는다.

명시적으로 Execution Cancel API를 호출한다.

115. Approval UX

Approval 화면에는 최소 다음을 보여준다.

- Agent/Workflow
- Action
- Target
- Risk
- Payload Summary
- Expiration

사용자가 무엇을 승인하는지 알 수 있어야 한다.

116. Error UX

사용자에게 Internal Stack Trace를 노출하지 않는다.

Stable Error Code를 사용자 친화적 메시지로 Mapping한다.

117. Retry UX

네트워크 실패 시 동일 Execution 생성 요청을 재전송할 경우 Idempotency Key를 재사용한다.

새 Key를 만들면 중복 실행될 수 있다.

118. API Client Timeout

Resource CRUD와 Agent Execution의 Client Timeout을 다르게 설정한다.

장기 Execution 자체는 Async Pattern을 사용한다.

119. Local Runtime

Local Runtime의 목적:

- 빠른 Agent 개발
- Workflow Test
- Mock Model
- Mock Plugin
- Trace

Local Runtime이 Production Policy를 모두 완벽히 재현한다고 가정하지 않는다.

120. Mock Model

Mock Model은 다음 테스트에 유용하다.

- Structured Output
- Tool Call
- Error
- Latency
- Retry

실제 AI 품질 Evaluation에는 Real Model이 필요하다.

121. Mock Plugin

Mock Plugin은 External Side Effect 없이 Workflow/Agent Integration을 테스트하는 데 사용한다.

122. Local Trace

개발자는 다음을 확인할 수 있어야 한다.

- Agent Action
- Tool Call
- Workflow Step
- Model Request Metadata
- Errors

Raw Chain-of-Thought를 디버깅 전제로 하지 않는다.

123. Test Pyramid

권장:

E2E

Integration

Contract

Component

Unit

AI 기능은 별도 Evaluation Layer를 추가한다.

124. Unit Test

대상:

- State Transition
- Validation
- Mapping
- Permission Logic
- Retry Decision

외부 Provider 실제 호출 없이 실행 가능해야 한다.

125. Contract Test

대상:

- API
- Event
- Provider Adapter
- Plugin
- Connector
- SDK

Contract 변경 시 자동으로 실패하도록 한다.

126. Integration Test

예:

Agent Runtime

+

Mock Model

+

Mock Plugin

또는:

Knowledge Ingestion

+

Vector Index

+

Retrieval

127. Runtime Failure Test

중요:

- Worker Crash
 - Queue Redelivery
 - Duplicate Task
 - Timeout
 - Cancellation
 - Checkpoint Recovery
-

128. Tenant Isolation Test

모든 Tenant-owned Domain에서 필수 테스트를 작성한다.

예:

Tenant A credential

+ Tenant B resource ID

→ denied

129. Security Test

개발자 수준에서도 최소:

- Unauthorized Access
- Privilege Escalation
- Secret Leakage
- Injection
- Invalid Signature
- SSRF where applicable

을 고려한다.

130. AI Evaluation Dataset

Agent 품질 테스트는 Versioned Dataset을 사용한다.

Dataset Example:

```
{  
  "input": "...",  
  "expected": {  
    "tool": "item.lookup"  
  }  
}
```

131. Evaluation 반복성

확률적 Model은 한 Test Case를 여러 번 수행할 수 있다.

Pass Rate와 Variance를 함께 볼 수 있다.

132. Evaluation과 Unit Test 차이

Unit Test:

항상 deterministic하게 통과/실패

Evaluation:

품질 Score 및 통계적 비교

둘을 동일 Test Runner 의미로만 처리하지 않는다.

133. Baseline

Production Agent Version을 Baseline으로 유지한다.

Candidate가 평균적으로 나빠지면 최신 Model을 사용하더라도 Promote하지 않는다.

134. Cost Evaluation

AI 평가에 다음도 포함한다.

- Input Tokens
- Output Tokens
- Model Calls
- Runtime Duration

품질이 동일하다면 비용이 낮은 Candidate가 더 나을 수 있다.

135. Performance Test

개발 단계에서 다음을 측정한다.

- API Overhead

- Queue Delay
- Runtime Scheduling
- Model Gateway Overhead
- Retrieval
- Plugin

외부 Provider Latency와 AGEX 자체 Latency를 분리한다.

136. Load Test

대표 시나리오:

- 다수 동시 Agent Execution
 - Workflow Fan-out
 - Knowledge Bulk Ingestion
 - Streaming
 - Plugin Burst
-

137. Code Review

Review Checklist:

- Domain Boundary
- Tenant Context
- Permission
- Error
- Retry
- Idempotency
- Logging
- Tests

- Version Compatibility
-

138. Critical Code Review

다음 변경은 Senior/Domain Owner Review를 요구해야 한다.

- Runtime State
 - Authorization
 - Tenant Isolation
 - Database Migration
 - Secret
 - Model Routing
 - Plugin Sandbox
-

139. Database Migration

Migration 순서:

Expand

→ Compatible Code

→ Backfill

→ Verify

→ Contract

기존 Code가 실행 중인 상태에서 즉시 Column/Table을 삭제하지 않는다.

140. Backfill

대량 Backfill은 일반 Request Transaction 안에서 수행하지 않는다.

비동기 Operation과 Progress Tracking을 사용한다.

141. Migration Test

Production 유사 Data Volume에서 Migration과 Rollback/Forward Recovery를 검증한다.

142. Error Model 개발

새 Domain Error는 Stable Error Code를 정의한다.

예:

AGENT_NOT_ACTIVE

PLUGIN_PERMISSION_DENIED

MODEL_CAPABILITY_UNAVAILABLE

String Message Parsing을 Client Contract로 사용하지 않는다.

143. Retryable Flag

내부 Error Type에는 Retry 가능 여부를 명확히 표현할 수 있다.

그러나 Client에 항상 이 필드를 그대로 노출할 필요는 없다.

144. Logging

Structured Logging 예:

```
{  
  "event": "plugin_action_failed",  
  "execution_id": "exe_123",  
  "plugin_id": "plg_123",  
  "error_code": "TIMEOUT"  
}
```

145. Logging 금지

다음은 Log에 포함하지 않는다.

- Prompt 전체
- Password
- Token
- API Key
- Secret
- Full Memory

필요한 경우 Metadata/Hash를 사용한다.

146. Metrics

새 Runtime Feature에는 Metric을 같이 설계한다.

예:

- success_total
- failure_total
- duration
- queue_wait

Observability를 개발 완료 후 추가하지 않는다.

147. Trace

Cross-Service 호출에서 Trace Context를 전달한다.

Agent → Model → Plugin 경로를 연결할 수 있어야 한다.

148. Audit

일반 Log와 Audit을 구분한다.

고위험 Action은 Domain Logic에서 Audit Event를 명시적으로 생성한다.

149. Security by Design

개발 시 다음 질문을 항상 확인한다.

- 누가 호출하는가.
- 어느 Tenant인가.
- 무엇에 접근하는가.
- 어떤 Side Effect인가.
- 어떤 Secret이 필요한가.
- 실패 시 어떤 영향인가.

150. Default Deny

새 Permission Type을 추가했는데 Policy 정의가 없으면 기본 Permit으로 하지 않는다.

151. Server-Side Permission

Frontend에서 Button을 숨겼다고 Security가 구현된 것은 아니다.

API/Runtime이 실제 Permission을 검증한다.

152. Secret Reference

예:

```
{  
  "credential_ref": "sec_123"  
}
```

다음은 금지:

```
{  
  "api_key": "sk-..."
```

}

공식 Resource에 평문 Secret을 저장하지 않는다.

153. Prompt Injection Defense

Agent 개발자는 User Input, Knowledge, Tool Output을 별도의 Untrusted Context로 취급한다.

이 Data가 System Instruction Layer를 변경하지 못하도록 한다.

154. Output Validation

AI가 생성한 다음 값은 반드시 검증한다.

- URL
- Resource ID
- Tool Input
- Enum
- Structured Payload

Model이 생성했다고 신뢰하지 않는다.

155. SSRF Defense

Plugin/Webhook/Connector가 URL을 입력받는 경우:

- Scheme
- Host
- Private IP
- Redirect

를 검증한다.

156. File Handling

업로드 처리 시:

- Type
- Size
- Malware
- Parsing
- Storage

검증을 수행한다.

File Name이나 MIME만 신뢰하지 않는다.

157. Package Security

Plugin/Marketplace Package 생성 시:

- Dependency Scan
- SBOM
- Signature
- Reproducible Metadata

를 포함할 수 있다.

158. Package Version

Published Package는 Immutable하다.

Fix가 필요하면 새 Version을 만든다.

159. Semantic Versioning

공개 SDK/Plugin Contract에는 Semantic Versioning을 적용할 수 있다.

- Major: Breaking

- Minor: Additive
- Patch: Compatible Fix

실제 AGEX Version Policy를 따른다.

160. Backward Compatibility

새 SDK/API가 기존 Resource Version을 처리할 수 있는지 테스트한다.

장기 Workflow 때문에 Runtime Compatibility가 중요하다.

161. Deprecated API 사용

새 Code에서는 Deprecated API를 사용하지 않는다.

기존 사용은 Migration Backlog를 만든다.

162. Feature Flag

실험 기능은 Feature Flag를 사용할 수 있다.

필수:

- Owner
- Scope
- Expiration

Security를 끄는 Feature Flag는 만들지 않는다.

163. Experimental Capability

실험 기능은 Production GA Capability와 명확히 구분한다.

예:

- experimental
- preview

- beta

Application이 안정 Contract로 오해하지 않도록 한다.

164. AI Coding Agent 활용

AI Coding Agent는 다음에 활용할 수 있다.

- Boilerplate
 - Test
 - Refactor
 - Documentation
 - Code Search
-

165. AI Coding Agent 금지 위임

Human Review 없이 다음을 맡기지 않는다.

- Tenant Isolation
 - Authorization
 - Runtime State Machine
 - Secret Management
 - Production Migration
 - Security Policy
-

166. AI 생성 코드

AI 생성 코드도 동일한:

- Review
- Tests
- Security Scan

- Contract Validation

을 통과해야 한다.

167. Developer Console 개발

Console은 공식 AGEX API를 사용한다.

Console Backend에서 Core DB를 직접 접근해 API 기능을 우회하지 않는다.

168. Workflow Editor 개발

Visual Editor의 저장 형식은 공식 Workflow Schema와 일치해야 한다.

UI 전용 별도 Workflow Format을 만들지 않는다.

169. Agent Builder 개발

Builder는 Agent Resource를 편집하는 UI다.

UI 내부 State가 Product Source of Truth가 아니다.

170. Execution Debugger 개발

Debugger는 다음을 보여준다.

- Timeline
- State
- Actions
- Tool Calls
- Knowledge Refs
- Policy Decisions
- Errors

Raw CoT 전체를 보여주지 않는다.

171. SDK 개발 원칙

SDK는 다음을 제공한다.

- Auth
- Tenant Context
- Typed Resources
- Pagination
- Error Mapping
- Streaming
- Idempotency Support

Server Security를 Client SDK에 의존하지 않는다.

172. Generated Client

API Client는 OpenAPI/Schema에서 생성할 수 있다.

Generated Layer와 Developer-friendly Wrapper를 분리하는 것이 좋다.

173. SDK Retry

SDK의 자동 Retry는 제한적으로 사용한다.

Permission/Validation Error를 자동 Retry하지 않는다.

174. SDK Idempotency

중복 위험 Operation은 SDK가 Idempotency Key 설정을 쉽게 제공해야 한다.

단, Client가 업무 단위 Key를 관리할 수 있도록 한다.

175. SDK Streaming

Streaming API가 끊기면 SDK가 Execution을 자동 Cancel하지 않는다.

최종 Execution 상태를 조회할 수 있는 API를 제공한다.

176. CLI 개발

CLI는 자동화 친화적으로 설계한다.

지원:

- machine-readable JSON output
- stable exit code
- non-interactive auth where permitted

Human-friendly Output과 Script Output을 구분할 수 있다.

177. CLI Secret

CLI Argument로 Secret을 직접 전달하면 Shell History에 남을 수 있다.

Secret 입력은:

- stdin
- secret file
- secret manager

방식을 고려한다.

178. CLI Confirmation

파괴적 명령:

tenant delete

plugin block

에는 Interactive Confirmation 또는 --force와 별도의 강한 조건을 요구할 수 있다.

179. API Mock

Frontend나 Partner 개발을 위해 공식 Schema 기반 Mock Server를 제공할 수 있다.

Mock 응답을 수기로 장기 유지하지 않는다.

180. Contract Drift 방지

CI에서 다음을 검증한다.

Schema

↔ API Server

↔ Generated SDK

불일치 시 Build를 실패시킨다.

181. Documentation as Code

Public API, SDK, Plugin Contract 변경 시 Documentation 업데이트를 같은 PR 또는 Release Process에 포함한다.

182. README와 공식 문서

Repository README는 빠른 시작용으로 사용한다.

Architecture Source of Truth를 README에만 두지 않는다.

183. ADR

중요 결정은 ADR로 남긴다.

예:

- Queue 선택
- Runtime Lease Model
- Vector Store

- Plugin Sandbox
-

184. ADR 내용

- Context
- Options
- Decision
- Consequences
- Alternatives

현재 구현만 적는 문서가 아니다.

185. Development Workflow

권장:

WBS Task

→ Design/Contract

→ Branch

→ Implementation

→ Tests

→ Review

→ Merge

→ Integration

186. Branch

구체 Branch Model은 Team 정책을 따르되 Long-Lived Divergent Branch를 최소화한다.

187. Pull Request

PR에는 최소:

- 무엇을 바꾸는가.
- 어떤 Contract가 바뀌는가.
- Test는 무엇인가.
- Migration이 있는가.
- Security 영향은 무엇인가.

를 기록한다.

188. PR 크기

가능하면 작은 논리 단위로 유지한다.

Runtime/Schema/API/UI 전체를 한 PR에 과도하게 넣어 Review가 불가능해지지 않도록 한다.

단, Contract를 깨는 분할도 피한다.

189. Merge Gate

최소:

- CI
 - Required Review
 - Tests
 - Contract Validation
 - Security Scan
-

190. CI 기본 Pipeline

Lint

→ Unit Test

→ Schema Validation

→ Contract Test

→ Security Scan

→ Build

필요 시 Integration Test를 추가한다.

191. Local 실패 재현

Production Bug Fix는 가능한 한 먼저 자동 Test로 재현한다.

Fix 후 해당 Test가 Regression Guard가 되어야 한다.

192. Feature 완료 기준

Developer 관점의 Done:

1. Contract
2. Code
3. Tests
4. Tenant
5. Security
6. Error
7. Metrics
8. Docs
9. Review

Code Merge만으로 완료가 아니다.

193. Production Publish 전

Agent/Workflow/Plugin 등 실행 Resource는 다음을 확인한다.

- Owner
 - Version
 - Dependencies
 - Permissions
 - Evaluation
 - Error Policy
 - Cost
 - Runtime Compatibility
-

194. Agent Production Checklist

- Goal 명확
 - Model Policy 유효
 - Tool 최소권한
 - Knowledge ACL 확인
 - Memory Scope 확인
 - Loop Limit 존재
 - Cost Budget 존재
 - Evaluation Pass
 - Kill/Suspend 가능
-

195. Workflow Production Checklist

- 모든 Step Reachable
- Loop bounded
- Retry bounded
- Timeout 설정

- Approval 고위험 Action 적용
 - Compensation 검토
 - Dependency Version 고정
 - Recovery Test
-

196. Plugin Production Checklist

- Manifest valid
 - Schemas valid
 - Permissions 최소
 - Side Effects 선언
 - Credential Ref
 - Network 제한
 - Timeout
 - Retry
 - Idempotency
 - Sandbox
 - Security Scan
-

197. Model Adapter Production Checklist

- Capability 정확
- Structured Output Test
- Streaming Test
- Tool Call Test
- Error Mapping
- Usage

- Health
 - Rate Limit
 - Security/Data Policy
-

198. Knowledge Connector Production Checklist

- Source Identity
 - Credential
 - Incremental Sync
 - Delete Handling
 - Classification
 - Provenance
 - Retry
 - Rate Limit
-

199. Developer Troubleshooting — Agent

확인:

1. Agent state
 2. Version
 3. Model Profile
 4. Permission
 5. Knowledge
 6. Tools
 7. Execution trace
 8. Error code
-

200. Developer Troubleshooting — Workflow

확인:

1. Current Step
 2. Variable Mapping
 3. Waiting Reason
 4. Approval
 5. Retry
 6. Dependency Version
 7. Timeout
-

201. Developer Troubleshooting — Plugin

확인:

1. Installation enabled
 2. Permission
 3. Credential
 4. Network
 5. Input Schema
 6. External API
 7. Output Schema
-

202. Developer Troubleshooting — Model

확인:

1. Capability
2. Provider Health
3. Data Classification

4. Tenant Policy
 5. Rate Limit
 6. Fallback
-

203. Developer Troubleshooting — Knowledge

확인:

1. Collection active
 2. Source indexed
 3. ACL
 4. Metadata filter
 5. Embedding/index version
 6. Retrieval score
-

204. Developer Troubleshooting — Memory

확인:

1. Scope
 2. Owner
 3. Retention
 4. Permission
 5. Deleted/expired state
 6. Ranking
-

205. 개발 금지사항

다음 구현은 허용하지 않는다.

- Agent에서 Provider SDK 직접 호출

- Runtime 없이 Agent Loop 장기 실행
- Workflow State를 Process Memory에만 저장
- Tenant ID를 Client 입력만 믿고 처리
- Cross-Domain DB 직접 접근
- Published Resource 직접 수정
- Tool Input을 무검증 실행
- AI Output을 Permission으로 해석
- Secret을 Prompt에 삽입
- Plugin Credential을 Agent에게 전달
- 모든 오류 무제한 Retry
- 무제한 Agent Loop
- 무제한 Workflow Loop
- Search 후 Tenant Filter
- Memory와 Knowledge를 하나의 비정형 Store로 무구분 운영
- Raw Chain-of-Thought 의존 Debugger
- Webhook Signature 미검증
- execute(anything) Plugin Action
- SDK에만 Security Enforcement 구현
- Test 없이 Production Publish
- AI Coding Agent 출력을 Review 없이 Merge
- Production DB 직접 수정 기반 운영 기능 구현
- Marketplace Package 실행을 Entitlement만으로 허용
- Security Policy를 비용 절감 목적의 Feature Flag로 비활성화

206. Developer Guide Acceptance Criteria

본 Developer Guide는 최소 다음을 충족해야 한다.

1. 개발 환경 구성 방식을 설명한다.
2. Resource 개발 순서를 정의한다.
3. Schema/Contract First를 적용한다.
4. Tenant Context 구현 원칙을 설명한다.
5. Agent 개발 방법을 설명한다.
6. Agent Instruction Hierarchy를 정의한다.
7. Agent Tool/Permission을 구분한다.
8. Context Builder를 설명한다.
9. Knowledge Binding을 설명한다.
10. Memory Proposal을 설명한다.
11. Workflow 개발 방법을 설명한다.
12. Durable Wait/Recovery를 설명한다.
13. Plugin 개발 방법을 설명한다.
14. Plugin Side Effect와 Credential을 정의한다.
15. Model Provider Adapter를 설명한다.
16. Knowledge Connector 개발을 설명한다.
17. Event/Webhook 개발을 설명한다.
18. Application Integration 방식을 설명한다.
19. Local Runtime을 설명한다.
20. Unit/Contract/Integration/AI Evaluation을 구분한다.
21. Security Coding 원칙을 포함한다.
22. Database Migration 원칙을 포함한다.
23. Observability 구현을 포함한다.
24. SDK/CLI 개발 원칙을 포함한다.

25. Production Checklist를 포함한다.

207. AGEX 개발자 핵심 원칙 요약

AGEX 개발자가 반드시 기억해야 할 핵심은 다음과 같다.

첫째

Agent는 Prompt가 아니라 Versioned Execution Resource다.

둘째

Workflow는 UI Flow가 아니라 Durable State Machine이다.

셋째

Model은 Agent가 직접 고르는 외부 SDK가 아니라 Model Gateway를 통해 제공되는 Capability다.

넷째

Plugin은 단순 API Wrapper가 아니라 Permission·Credential·Side Effect·Sandbox가 포함된 Extension Contract다.

다섯째

Knowledge와 Memory는 동일한 Vector Storage 개념이 아니다.

여섯째

AI가 Action을 제안할 수 있지만 Runtime만 Action을 실제로 승인하고 실행한다.

일곱째

Tenant, Security, Audit, Cost, Observability는 기능 개발 이후 붙이는 부가기능이 아니다.

208. Reference Development Flow

일반적인 AGEX Feature 개발은 다음 흐름을 따른다.

Requirement

↓

Domain Identification



Resource / Contract Design



Schema



Permission / Tenant / Policy



Implementation



Unit Test



Contract Test



Integration



Security



Observability



Documentation



Evaluation / Acceptance



Publish

209. Agent Application 전체 예시 흐름

Application Request



AGEX API



Execution Created



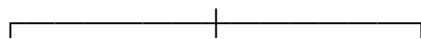
Runtime



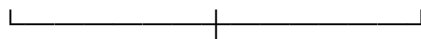
Agent



Context Builder



Knowledge Memory Policy



Model Gateway



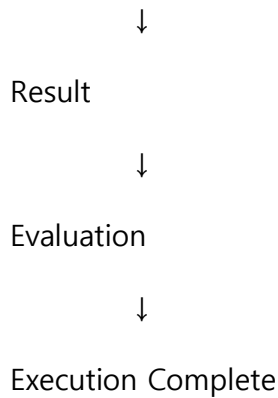
Action Candidate



Validation



Plugin Tool



개발자는 이 흐름을 개별 Application 코드에 다시 구현하지 않는다.

210. Developer Guide의 최종 정의

AGEX Developer Guide는 개발자가 LLM API를 호출하는 방법을 설명하는 문서가 아니라 Agent, Workflow, Knowledge, Memory, Plugin 및 Model을 AGEX의 공식 Resource와 Runtime Contract 위에서 안전하게 구현하는 방법을 규정하는 개발 기준이다.

AGEX 개발자는 먼저 기능이 어느 Domain에 속하는지를 식별하고, Resource와 Canonical Schema, Permission, Tenant Scope 및 Lifecycle을 정의한 뒤 실제 Code를 작성해야 한다.

Agent는 Prompt 문자열의 집합이 아니라 Goal, Instruction, Capability, Permission, Model Policy, Knowledge, Memory 및 Tool을 결합한 Versioned Resource이며 모든 Action은 Runtime Validation을 거쳐야 한다.

Workflow는 Application Process Memory에 상태를 보관하는 흐름 제어가 아니라 Wait, Approval, Retry, Timeout, Checkpoint 및 Recovery를 지원하는 Durable Execution Graph로 구현한다.

Knowledge는 출처·ACL·Citation을 보존하고, Memory는 Scope·Retention·Delete·Confidence를 가진 별도의 Persistence Domain으로 구현한다.

Plugin은 External API를 단순히 Wrapping하는 코드가 아니라 Manifest, Capability, Action Schema, Permission, Side Effect, Credential Broker, Network Policy 및 Sandbox를 포함하는 Extension Package다.

Model Provider는 Agent 내부에 직접 결합하지 않으며 공통 Provider Adapter를 통해 Model Gateway에 연결하고 Capability, Usage, Error, Streaming 및 Health를 표준화한다.

모든 Application Integration은 AGEX API 또는 SDK를 사용하고, Browser·Application·CLI

가 Core Database나 Provider를 직접 호출하여 Platform Security와 Runtime을 우회해서
는 안 된다.

개발 완료는 Code가 동작하는 시점이 아니라 Tenant Isolation, Security, Tests,
Observability, Documentation 및 필요한 AI Evaluation까지 통과한 시점으로 판단한다.

AGEX Developer Guide는 다음 문장으로 최종 정의한다.

**AGEX Developer Guide는 AI 기능을 빠르게 연결하는 개발 매뉴얼이 아니라, AI의 확률
적 판단을 안정적인 Resource Contract·Durable Runtime·Security·Tenant·Governance
안에서 실제 Product Code로 구현하기 위한 AI Operating System의 공식 개발 기준이
다.**

본 문서는 AGEX의 Agent, Workflow, Knowledge, Memory, Plugin, Model Adapter,
Application Integration, Testing, SDK 및 Production Development 방식의 공식
Developer Guide로 사용한다.

211. AGEX 공식 문서 세트 완료

본 **AGEX Developer Guide**를 마지막으로 현재 정의된 AGEX 공식 문서 30종이 모두 작
성되었다.

완료된 공식 문서 체계는 다음과 같다.

1. Product Vision
2. Product Philosophy
3. Product Architecture
4. Core Architecture
5. Runtime Architecture
6. Agent Framework
7. Workflow Engine
8. Knowledge Engine
9. Memory Engine
10. Plugin Framework
11. Marketplace

12. SDK
13. API Specification
14. Security Architecture
15. Multi-Tenant Architecture
16. Deployment Architecture
17. Governance
18. Product Roadmap
19. Development Roadmap
20. WBS
21. Development Plan
22. Resource Plan
23. Cost Estimation
24. Business Model
25. Technical White Paper
26. Product Documentation
27. Operations Guide
28. API Guide
29. Administrator Guide
30. **Developer Guide**